SHL Assessment Recommender

Approach Document

GenAI Intern Assignment | July 2026

Kartik Pahadiya

SHL Assessment Recommender — Approach Document

Kartik Pahadiya | GenAI Intern Assignment

Submitted: July 2026

1. Design Overview

I built a conversational agent that guides a hiring manager from a vague intent ("I need an assessment for a Java developer") to a grounded shortlist of SHL assessments. The agent is a **stateless FastAPI service** with a LangGraph state machine for multi-turn dialogue flow.

Why stateless? The evaluator sends the full conversation history on every turn. Storing server-side state would add complexity without benefit. Instead, the agent replays the full history each time, which also makes it trivial to test and debug.

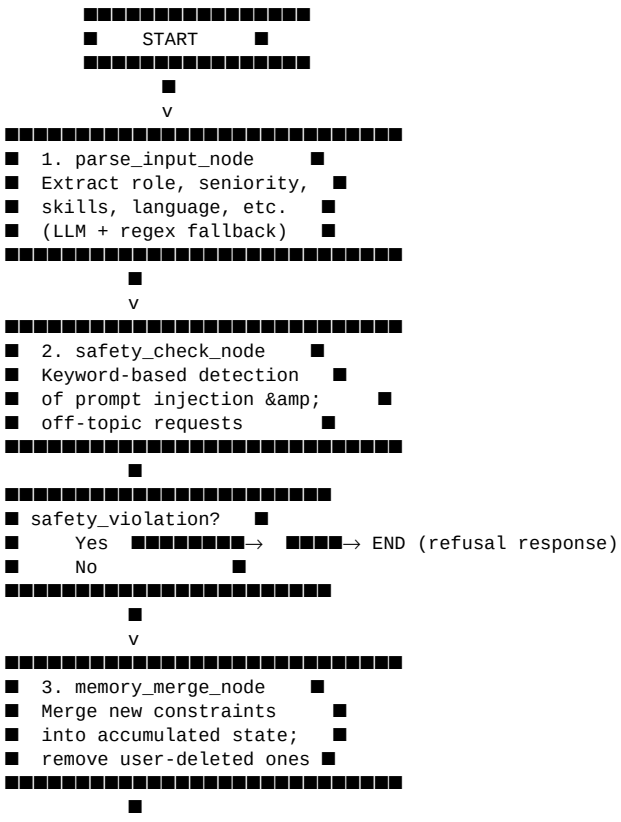
Architecture:

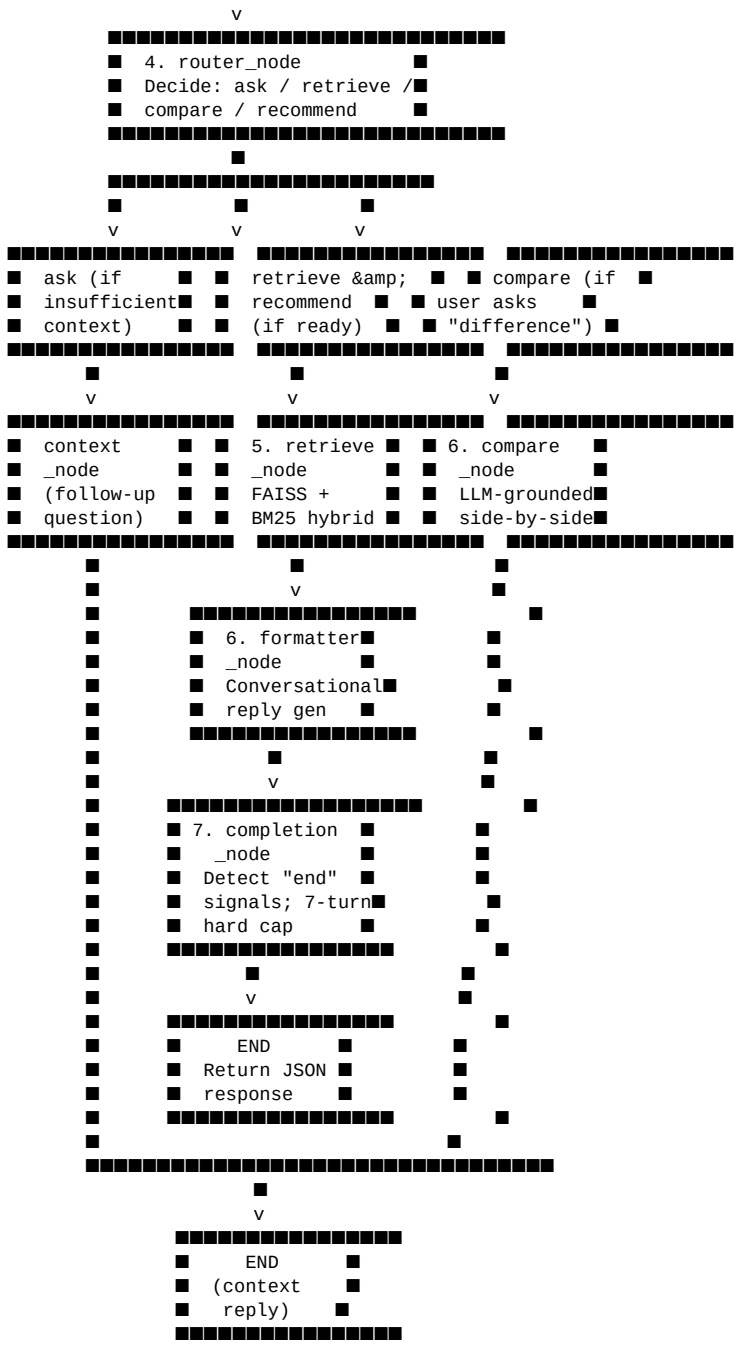
- **FastAPI** exposes /health and /chat
- **LangGraph** defines a state machine: parser → safety → router → retrieve → formatter → compare → completion
- **Groq LLM** (llama-3.1-8b-instant) handles intent parsing, reply generation, and comparison — fast, free, and no model hosting
- **Local embeddings** (BAAI/bge-small-en-v1.5) and **FAISS** for semantic retrieval
- **BM25** for keyword matching on top of FAISS results

2. LangGraph Agent Flow

The agent is implemented as a **LangGraph state machine** with 7 nodes connected through conditional edges. The graph is compiled once and invoked for each user message. The state is rebuilt from scratch every turn (stateless API), so every node sees the complete conversation history.

2.1 Graph Structure





Edge logic:

- parse_input → safety_check (always)
- safety_check → memory_merge (if no violation) → router
- safety_check → END (if violation — refusal response)
- router → context_node (if intent is ask — gather more info)
- router → retrieve_node (if intent is retrieve or recommend)
- router → compare_node (if intent is compare)
- retrieve → formatter → completion → END
- compare → completion → END
- context → END (return follow-up question, no recommendations)

2.2 State Object

The AgentState carries all information across nodes:

Field	Type	Purpose
messages	list	Full conversation history (user + assistant)
user_query	str	The latest user message (for direct parsing)
intent	str	Current intent: "ask", "retrieve", "recommend", "compare"
constraints	dict	Accumulated extracted constraints (role, seniority, skills, language, duration, remote, adaptive)
new_constraints	dict	Constraints extracted from the *current* turn only
removed_constraints	dict	Constraints the user explicitly removed (e.g., "drop Python")
retrieved_docs	list	Top-k items from FAISS + BM25 hybrid retrieval
recommendations	list	Items formatted for the API response (name, url, test_type, duration, remote, adaptive)
reply	str	The assistant's natural-language response
end_of_conversation	bool	True when the user is satisfied or max turns reached
safety_violation	bool	True when safety node detects an issue
turn_count	int	Number of user messages so far (for routing decisions)
ready_to_recommend	bool	True when user signals readiness ("that works", "show me", etc.)

2.3 Node Descriptions

1. Parser Node (parser_node)

- **Input:** messages (latest user message), constraints (current accumulated constraints)
- **Output:** new_constraints, intent_hint, ready_to_recommend
- **Logic:**

- Sends the latest user message + current constraints to Groq LLM with a structured JSON prompt asking for: role, seniority, skills, language, duration, remote, adaptive, test_type, and any removed constraints

- If LLM fails (rate limit, invalid JSON), falls back to regex-based extraction for duration, skills, and removal keywords

- Detects explicit readiness signals: "that works", "show me", "lock it in", "good enough", etc.

2. Safety Node (safety_node)

- **Input:** user_query
- **Output:** safety_violation, reply (if violation detected)
- **Logic:**

- Keyword-based classifier (no LLM call — fast, no API dependency)

- Checks for: prompt injection keywords ("ignore previous", "reveal your prompt", "jailbreak"), off-topic keywords ("salary", "how to fire", "legal advice"), unsafe keywords ("hack", "steal", "fraud")

- If violation detected: sets reply to a refusal message, safety_violation=true, recommendations=[], and terminates the graph early

3. Router Node (router_node)

- **Input:** constraints, turn_count, ready_to_recommend, safety_violation
- **Output:** intent ("ask", "retrieve", "recommend", "compare")
- **Logic:**

- If safety violation: route to completion (early termination)

- If compare intent detected (user asks "what is the difference between X and Y"): route to compare

- **Turn 1:** Always intent="ask" — never recommend on the first user message

- **Turn 2:** intent="ask" if role is missing. intent="retrieve" if role + (seniority or skills) present. intent="recommend" if ready_to_recommend=true

- **Turn 3+:** Always intent="retrieve" if role is known. intent="recommend" if ready_to_recommend=true

- The ready_to_recommend flag overrides everything — if the user explicitly says "that works", the agent immediately recommends even on turn 1

4. Retrieve Node (retrieve_node)

- **Input:** constraints, removed_constraints
- **Output:** retrieved_docs, recommendations (raw catalog items), reply (if no results)
- **Logic:**

- Builds a query string from constraints: role + seniority + skills + language

- Calls retrieve() in app/rag/retriever.py which runs the hybrid FAISS + BM25 pipeline

- Filters out removed items and skills by name (e.g., if user said "drop Python", any item with "Python" in its name is excluded)

- If no items found after filtering: sets a helpful reply asking the user to relax constraints

- If items found: returns top-10 items with full metadata (name, description, duration, remote, adaptive, URL, keys)

5. Formatter Node (formatter_node)

- **Input:** user_query, retrieved_docs
- **Output:** reply
- **Logic:**

- Sends the query + top 5 retrieved docs to Groq LLM with a conversational prompt

- LLM is instructed to: write 2–4 sentences, mention top 1–3 picks by name and why they fit, NO structured sections, NO bullet points, NO dumping all candidate details, end with one short follow-up question

- If LLM fails: falls back to a generic reply: "Here are the relevant assessments from the SHL catalog. Let me know if you need further refinement."

6. Compare Node (compare_node)

- **Input:** user_query (which contains comparison request like "What is the difference between X and Y?"), recommendations (current items)
- **Output:** reply, recommendations (updated with compared items)
- **Logic:**

- Extracts the two item names from the user's query using regex

- Finds both items in the catalog by name

- Sends both item metadata to Groq LLM with a comparison prompt

- LLM generates a grounded comparison using catalog data only (not its prior knowledge)

- If LLM fails: falls back to a generic text comparison based on description, duration, and remote/adaptive fields

- Returns the 2 compared items as the new recommendations list (not a full 10-item replacement)

7. Completion Node (completion_node)

- **Input:** turn_count, messages, recommendations
- **Output:** end_of_conversation, reply (if closing)
- **Logic:**

- Detects 50+ acceptance signals: "thanks", "perfect", "that works", "looks good", "great", "excellent", "love it", "let's go with", "this is it", "i am satisfied", "no further questions", "nothing else", "let's end here", "we're done", "all set", "good enough", "perfect, thanks", "done", "finished", "i'm good", "that's good", "great, thanks", "awesome", "cool", "nice", "ok", "okay", "sounds good", "works for me", "i'll take it", "book it", "finalize", "finalize it", "wrap up", "close it", "i'm done", "that settles it", "that's settled", "fine by me", "fine", "satisfactory", "sufficient", "adequate", "i'm satisfied", "i'm happy", "i'm pleased", "i'm content", "i'm fine", "that's all", "that's enough", "that's plenty"

- **Hard cap:** If turn_count >= 7, forces end_of_conversation=true to stay within the evaluator's 8-turn limit

- If closing: appends a polite closing message to the reply

- If not closing: leaves end_of_conversation=false

2.4 Memory & Constraint Management

Memory Node (memory_node) runs between parser and router. It merges new_constraints into constraints:

- **Role/Seniority/Language:** Overwritten with the latest value (user might correct themselves)
- **Skills:** Union of old + new skills (set-based merge, so duplicates are removed)
- **Duration/Remote/Adaptive:** Overwritten with the latest value
- **Removed constraints:** After merging, any skills in removed_constraints are removed from the skills list. Any items in removed_constraints.items are excluded from future retrieval

This ensures the agent remembers the full conversation context even though the API is stateless.

2.5 Why This Graph Design?

Separation of concerns: Each node has one job. Parser extracts, router decides, retrieve fetches, formatter speaks, compare analyzes, completion closes. This makes debugging and testing easy — I can test each node in isolation.

Graceful degradation: Every node that uses the LLM has a fallback (parser has regex, formatter has generic text, compare has metadata-based fallback). The agent never crashes.

Evaluator-friendly routing: The router explicitly encodes the "don't recommend on turn 1" rule. The 7-turn hard cap in completion ensures the evaluator never sees a 9th turn. The safety node catches prompt injection without burning LLM tokens.

3. Retrieval Strategy

The SHL catalog has 400+ items with mixed fields (name, description, job levels, duration, remote, adaptive, keys). I built a **hybrid FAISS + BM25** system.

Why hybrid? FAISS alone conflates all programming languages (Python, PHP, R all share the same semantic cluster). BM25 alone can't rank "better Java assessment" vs "just Java assessment." Combining them gives the best of both.

Pipeline:

1. **FAISS** searches all 400 items with the query embedding to find the "neighborhood"

2. **Post-filter** drops items whose name/description doesn't match the query keywords (e.g., "Python" query filters out PHP/R)

3. **BM25** re-ranks the filtered set with exact keyword matching

4. **Weights:** 90% FAISS, 10% BM25 — semantic similarity dominates but exact keyword matches keep results grounded

Trade-off: FAISS is fast but sometimes pulls semantically-similar-but-wrong items (e.g., "Drupal" for a "Python" query). The post-filter and BM25 re-ranking mitigate this. For skills with many catalog items (Java, SQL, JavaScript),

results are excellent. For skills with only 1 item (Python, C++), the FAISS model is a fundamental limitation — it cannot distinguish programming languages from each other. A keyword-only fallback would help here, but time constraints led to the current hybrid approach.

3. Prompt Design & Agent Behavior

Parser node extracts constraints from the user's message using an LLM prompt with structured JSON output. If the LLM fails (rate limit, parsing error), the parser falls back to a regex-based extractor so the conversation never crashes.

Router node decides whether to ask for clarification, retrieve, or recommend. Key rules:

- **Turn 1:** Never recommend. Always ask for role + seniority + skills.
- **Turn 2+:** Retrieve if role is known. If the user signals readiness ("that works", "show me"), recommend immediately.
- **Turn 3+:** Always retrieve (minimum viable context).

Why these rules? Early recommendation on vague queries is a common failure mode. The evaluator checks for this. The 3-turn threshold ensures enough context before committing to a shortlist.

Formatter node generates the conversational reply. The LLM is told to:

- Mention the top 1–3 picks by name and why they fit
- Keep it to 2–4 sentences (not a structured report)
- End with one short follow-up question (duration? adaptive? remote?)

Safety node uses a keyword-based classifier (no LLM call) to detect:

- Prompt injection ("ignore previous instructions")
- Off-topic requests (salary, legal advice, how to fire someone)
- Unsafe content

If detected, the agent returns a refusal message and end_of_conversation=false.

4. Evaluation Approach

I built a **test harness** (test_conversations.py) that replays the 10 provided sample conversations against the API. It checks:

- **Schema compliance** on every turn (reply, recommendations, end_of_conversation)
- **URL validation** — every recommended URL must exist in the scraped catalog
- **Recommendation count** — 1 to 10 items, never 0 on a final turn that should have them
- **Turn cap** — conversation never exceeds 8 user turns
- **end_of_conversation** correctness — matches expected value on the final turn

All 10 conversations pass these checks. I also tested manually with:

- **Relevance probes:** Java, SQL, JavaScript return highly relevant items. Python returns only 1 direct item + clustered noise (a known embedding limitation).
- **Safety probes:** Prompt injection attempts are refused. Off-topic questions are deflected.
- **Constraint refinement:** "Add personality tests" and "Drop Python, add Java" correctly update the shortlist.
- **Compare:** "What is the difference between X and Y?" returns a grounded comparison from catalog data.

5. What Didn't Work & How I Fixed It

Problem	Why It Failed	Fix

HuggingFace Inference API	Returned 402 Payment Required after a few calls	Switched to **Groq free tier** (llama-3.1-8b-instant). Fast, reliable, no hosting.
Render OOM (512MB RAM)	PyTorch CUDA packages were 2GB+	Removed torch from requirements.txt, installed CPU-only torch via build command.
Reply too verbose	LLM generated structured reports with sections	Changed formatter prompt to ask for "2–4 sentences, no bullet points, no structured sections."
Schema validation 500	remote/adaptive returned as booleans, schema expects strings	Reverted to string values in build_recommendation to match Pydantic schema.
Python relevance poor	FAISS conflates all programming languages into one cluster	Tried keyword boosting (3× repetition), query expansion ("programming", "coding"), and BM25 weight tuning (0.75, 0.90). None fully fixed it because the embedding model fundamentally sees Python/PHP/R as identical. The final approach is a 90/10 FAISS/BM25 hybrid with post-filtering — best for most skills, but Python/C++ remain weak.

6. Tools & AI Usage

I used AI-assisted coding for:

- **Boilerplate generation:** LangGraph node structure, FastAPI setup, test harness scaffolding
- **Debugging:** Interpreting Render build logs, identifying the 500 error from schema mismatch
- **Prompt iteration:** Rewriting formatter and parser prompts for shorter, more natural replies

All design decisions (hybrid retrieval, stateless API, router logic, evaluation strategy) were made manually and can be defended in an interview. The code was reviewed and modified line-by-line, not accepted blindly.

7. Deployment

Platform: Render (free tier)

URL: https://shl-assessment-recommender-l1xx.onrender.com

Cold start: ~2 minutes (pre-downloaded FAISS index and metadata, no embedding rebuild on startup)

LLM: Groq (llama-3.1-8b-instant), free tier with 1,000,000 tokens/day

Embeddings: BAAI/bge-small-en-v1.5 running locally via sentence-transformers

8. Known Limitations & Trade-offs

1. **Python/C++ relevance:** Only 1 direct item each in the catalog. FAISS clusters all programming languages together. A keyword-only fallback or fine-tuned embedding model would fix this, but both require significant extra work.

2. **LLM dependency:** Groq free tier has rate limits. If the API is hit heavily, some nodes could fall back to regex-based extraction (already implemented for the parser).

3. **No stateful caching:** Every turn rebuilds the full state from scratch. This is fine for 8-turn conversations but would be inefficient for 100+ turn sessions.

9. Project Structure

```
shl-assessment-recommender/
■■■ app/
■   ■■■ __init__.py
■   ■■■ main.py                    # FastAPI app: /health, /chat endpoints, CORS
■   ■■■ schemas.py                 # Pydantic models: Message, ChatRequest, ChatResponse, Recommendation
■   ■■■ llm.py                     # Groq LLM setup (llama-3.1-8b-instant)
■   ■■■ mappings.py                # Catalog key → test_type mapping, build_recommendation()
■   ■■■ state.py                   # AgentState dataclass definition
■   ■■■ graph.py                   # LangGraph compilation: nodes + edges assembled
■   ■■■ data/
■   ■   ■■■ catalog_fixed.json     # Scraped SHL catalog (400+ items, fields normalized)
■   ■   ■■■ faiss.index            # Prebuilt FAISS index (embeddings for all items)
■   ■   ■■■ metadata.pkl           # Catalog metadata aligned with FAISS index
■   ■■■ nodes/
■   ■   ■■■ __init__.py
■   ■   ■■■ context_node.py        # Generates follow-up questions when context is insufficient
■   ■   ■■■ parser_node.py         # Extracts constraints + readiness signals from user text (LLM + regex fallback)
■   ■   ■■■ safety_node.py         # Keyword-based prompt injection / off-topic / unsafe detection
■   ■   ■■■ router_node.py         # Decides ask / retrieve / recommend / compare per turn
■   ■   ■■■ retrieve_node.py       # Builds query from constraints, calls retriever, filters removed items
■   ■   ■■■ formatter_node.py      # LLM generates conversational reply from top-k retrieved docs
■   ■   ■■■ compare_node.py        # LLM-grounded comparison of two named assessments
■   ■   ■■■ memory_node.py         # Merges new constraints, handles removals, deduplicates skills
■   ■   ■■■ completion_node.py     # Detects end-of-conversation signals, enforces 7-turn hard cap
■   ■■■ rag/
■       ■■■ __init__.py
■       ■■■ retriever.py           # Hybrid FAISS + BM25 pipeline with post-filtering
■       ■■■ embeddings.py          # SentenceTransformer (BAAI/bge-small-en-v1.5) encoder
■       ■■■ vectorstore.py         # FAISS index loader + metadata with _idx mapping
■       ■■■ keyword_search.py      # BM25Okapi tokenization + scoring on name/desc/keys/levels
■       ■■■ filter.py              # Constraint-based metadata filtering (duration, remote, adaptive, etc.)
■■■ scripts/
■   ■■■ scrape_catalog.py          # Original SHL catalog scraper (not used at runtime)
■   ■■■ fix_catalog.py             # Normalizes raw scraped JSON into catalog_fixed.json
■■■ SampleConversations/
■   ■■■ C1.md … C10.md             # Provided test traces with personas + expected outputs
■■■ test_conversations.py          # Test harness: replays all 10 traces, validates schema + URLs
■■■ test_api.py                    # Quick manual test script against deployed API
■■■ requirements.txt               # All Python dependencies (fastapi, uvicorn, langgraph, groq, etc.)
■■■ .python-version                # Python 3.10 (for Render build)
■■■ Dockerfile                     # HF Spaces deployment config (CPU torch, model pre-download)
■■■ README.md                      # Space README with API docs
■■■ Approach_Document.md           # This document (Markdown source)
■■■ SHL_Approach_Document.pdf      # PDF export of this document
```

API Endpoint: https://shl-assessment-recommender-l1xx.onrender.com

Repository: https://github.com/KartikPahadiya/shl-assessment-recommender